



Detecting and preventing replay attacks in industrial automation networks operated with profinet IO

Steffen Pfrang¹ · David Meier¹

Received: 14 July 2017 / Accepted: 24 January 2018 / Published online: 24 February 2018
© Springer-Verlag France SAS, part of Springer Nature 2018

Abstract

Modern industrial facilities consist of controllers, actuators and sensors that are connected via traditional IT equipment. The ongoing integration of these systems into the communication network yields to new threats and attack possibilities. In industrial networks, often distinct communication protocols like *Profinet IO (PNIO)* are used. These protocols are often not supported by typical network security tools. In this work, we present two attack techniques that allow to take over the control of a PNIO device, enabling an attacker to replay previously recorded traffic. We model attack detection rules and propose an intrusion detection system (IDS) for industrial networks which is capable of detecting those replay attacks by correlating alerts from traditional IT IDS with specific PNIO alarms. As an additional effort, we introduce defense in depth mechanisms in order to prevent those attacks from taking effect in the physical world. Thereafter, we evaluate our IDS in a physical demonstrator and compare it with another IDS dedicated to securing PNIO networks. In a conceptual design, we show how network segmentation with flow control allows for preventing some, but not all of the attacks.

Keywords Industrial networks · Replay attacks · Intrusion detection and prevention · Attack detection modeling · Defense in depth

1 Introduction

Automation systems play a key role in modern industrial facilities and are therefore an important part of our economy. New technology tries to maximize the efficiency of such systems by simplifying the installation of automation systems, as well as providing the available data to the enterprise level. In recent years, this was achieved more and more by the introduction of well-known IT technology into areas where in the past dedicated and incompatible technologies were used. While automation and control systems still have certain requirements, new standards aim to provide these requirements using standard IT technology. One such standard is *Profinet IO (PNIO)*, as specified in [14]. This Industrial Ethernet standard has the possibility to connect various automation systems and provide them with real-time communication. PNIO, as most new automation standards, is

mostly concerned with providing reliable services and does not provide any distinct security mechanisms.

This situation, combined with the already mentioned process of networks getting connected to more and more services, renders the threat of cyberattacks into a realistic scenario. Even if there are no inherent security controls in PNIO,¹ it is crucial to be able to detect attacks targeting PNIO devices. Because such attacks require little to no knowledge about the industrial process being run and are easy to adopt, this makes them a rewarding target for any attacker.

In this work, we will show how the missing security in automation protocols can facilitate the ability to successfully attack control system equipment by using a replay attack method on PNIO. We will then show how it is possible to detect these attacks, taking multiple information sources as an input for an intrusion detection system (IDS). Taking into account the mechanisms of defense in depth (DiD), we then describe the ability to prevent some of those attacks by the means of network separation with flow control. To be able to understand the attack and proposed detection model, this

✉ Steffen Pfrang
steffen.pfrang@iosb.fraunhofer.de

David Meier
david.meier@iosb.fraunhofer.de

¹ Fraunhofer IOSB, Karlsruhe, Germany

¹ This means that such a PNIO network should ideally be operated in a completely separated environment without any link to the outside.

work also includes a brief overview of the relevant details of PNIO.

This is an extended version of the paper [21] presented at the ICISSP 2017 in Porto, Portugal which includes the following new and extended content:

- Description of defense in depth approach in section 2
- Both an attacker modeling and an extended environment setup in the case study section 3
- An updated description of the PNIO Detector in the Attack Detection Implementation section 4.2
- A new section 5 about the prevention strategy using methods of DiD
- An evaluation of the prevention strategy in section 6

2 Background and related work

In this section, we will give background information on the relevance of PNIO and the protocol specifications which are important for our study. The second part of this section is dedicated to the concept of DiD. In the third part, we will focus on approaches to security in automation protocols. Related work available in the area of control system security and on the security of PNIO is presented in the last section.

2.1 Profinet IO

According to a recent study [13], Ethernet-based fieldbuses have a market share of 38% within industrial automation systems, while classic fieldbuses have 58% and wireless variants 4%. The annual growth of Ethernet-based fieldbuses is estimated with 20%, whereas Siemens is the market leader in Europe and China with its “Profibus family” including PNIO [12]. In total, 21% of the sold Ethernet-based automation systems are running PNIO.

PNIO is a real-time enabled industrial Ethernet standard defined in IEC 61158-6-10. Figure 1 depicts the typical setup of a PNIO system. The standard defines three different roles: PNIO supervisor, PNIO controller and PNIO device.

The PNIO supervisor, for example a programming station, administrates the initial setup and the diagnosis of a PNIO system. The PNIO controller, typically a programmable logic controller (PLC), parametrizes connected PNIO devices and runs the process. A PNIO device is a field device which transforms virtual process data into physical process data and vice versa. It is able to signal critical system states.

In the following chapters, we will make use of an example scenario for a PNIO process. In this scenario, a PLC (as the PNIO controller) is operating a stepping motor (the PNIO device). The following subsections will provide technical details about the PNIO protocol behaviour relevant to the this scenario. Starting with the discovery and configu-

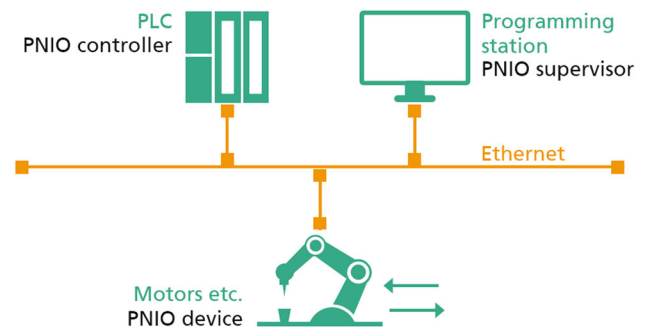


Fig. 1 A basic PNIO setup introducing three different roles

ration protocol DCP and the context management protocol CM, we describe the startup phase of a PNIO network like it happens when the power returns after an outage. The last subsection covers the alarm management in PNIO.

2.1.1 PNIO discovery and configuration

The Discovery and Configuration Protocol (DCP) is an Ethernet-based protocol within the PNIO protocol suite. Using the *DCP Set Request*, a controller is able to configure a PNIO device. Such a request can contain multiple configuration blocks, for example to set the name and the IP address using only a single packet. The request is then answered by a *DCP Set Response* that signals what configuration options were tried to be set and if the request could be executed. It does not contain the actual values from the *DCP Set Request*.

2.1.2 PNIO context management

CM (Context Management) is a UDP-based PNIO protocol to setup the application relation (AR) between PNIO controllers and devices. The controller is sending a *CM ConnReq* request, specifying various properties of the communication channel, including its communication relations (CR). These can be data inputs and outputs (IO), as well as properties for alarm handling. The properties of IO CR include values for the *DataHoldFactor*, *SendClockFactor* and *ReductionRatio*. From these values, the *DataHoldTime* of the respective CR is calculated, as shown in equation 1.

$$\begin{aligned}
 \text{DataHoldTime} &= \text{DataHoldFactor} \\
 &\times \text{SendClockFactor} \\
 &\times \text{ReductionRatio} \\
 &\times 31.25 \mu\text{s}
 \end{aligned} \tag{1}$$

A watchdog timer of that duration is set every time data is received for the respective CR. When this timer expires, it will trigger an *PN-RT* alarm frame with an appropriate error message, signaling that a timeout occurred.

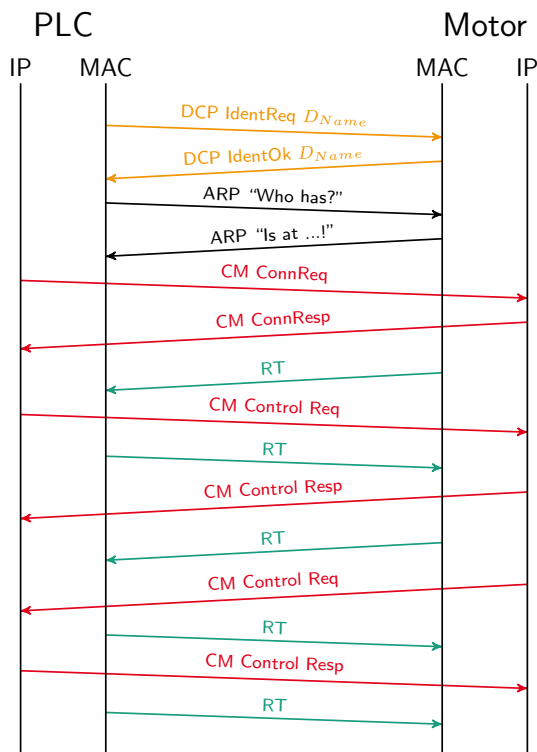


Fig. 2 The regular startup process of a PNIO network at the example of a PLC and a motor

The PNIO device replies to the *CM ConnReq* with a *CM ConnResp* response. This response can acknowledge the request or deny it, for example if there are no resources left for a new AR. In case of a successful connection request, the already established CRs will start exchanging data and the controller can proceed establishing further CMs. When all desired CMs are established, the controller will send a *CM Control Req* request signaling the end of parameter transmission, that will again be acknowledged by the PNIO device through a *CM Control Resp*. Afterwards, the device is sending a *CM Control Req* signaling application readiness. After the controller has acknowledged this request, the communication establishment is complete.

2.1.3 PNIO startup

When an industrial setup operated with PNIO starts up, the sequence of network packets shown in Figure 2 can be observed. We will focus on the connection establishment between the PLC (as PNIO controller) and the motor (as PNIO device). Additionally, we consider the case that all the components are configured for operation. That means, that the process is already running which is a reasonable assumption when dealing with real production environments.

From the Profinet protocol suite, there are three protocols involved: DCP, CM and RT (Real Time). Additionally, PNIO

makes use of ARP (Address Resolution Protocol). CM is an UDP-based protocol, while the other protocols are operating directly on the MAC layer (Ethernet).

The startup process is initiated by the PNIO controller. It asks with a multicast *DCP Ident Request* for a device with the PNIO name that has been configured initially in the PLC program. The PNIO device with that particular name replies to that request with an unicast *DCP Ident OK* that is directed to the PLC. The *Ident OK* message includes, amongst other information, the IP address configuration of that device.

Subsequently, the controller starts an *ARP request* for the IP address given by the PNIO device. If the device is not yet configured with that IP address, the controller sends a *DCP Set Request*, setting the IP address configuration of the PNIO device in order to be able to communicate with it using UDP for the next step, the PNIO-CM.

2.1.4 PNIO alarm management

The signaling of errors and warnings is also an integral part of PNIO. On the network level, this signaling is achieved by the usage of special *alarm frames*. There are multiple alarm types, depending on the origin and cause of the alarm. For example, *process alarms* are originating from the executed process on a field device, while *diagnosis alarms* are originating from the field device itself [23]. Only *process alarms* have their priority set to HIGH, while all other type have a LOW priority. Alarms are queued into a buffer according to their priority and only removed from the buffer once they are acknowledged. Every alarm is therefore assigned a sequence number which also needs to be part of the acknowledgment of the communication partner, i.e. the PNIO controller or device. This means other alarms of the same priority are not reported until all alarms prior to them are removed from the queue buffer. The alarm buffer is cleared when the application relation (AR) between controller and device is terminated, for example by a new connection request.

The PNIO standard holds a predefined list of alarm messages that can be extended by manufacturer specific messages. Alarms are sent within an acyclic *PN-RT* frame, the alarm frame (*PN-AL*), specifying the alarm type, alarm origin and structure of the alarm message.

2.2 Defense in depth

With regard to the threat of cyberattacks, countermeasures have to be deployed in the industrial IT domain. The traditional IT security approach is described in a multitude of reports and international standards like the ISO 27000 or IEC 62443 series. These countermeasures are part of a strategy to reduce the risks for all assets important for an organization, including not only technical equipment, but all kinds of

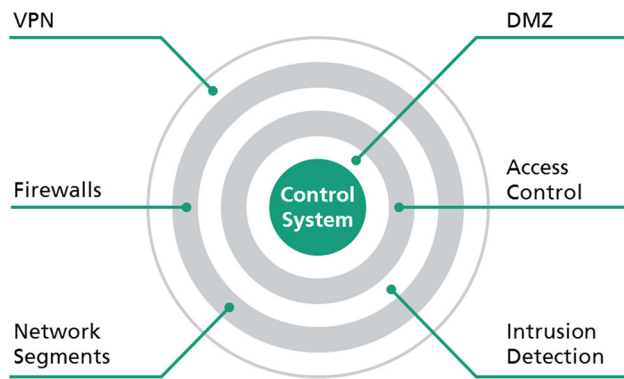


Fig. 3 Example of a defense in depth strategy and associated detection and prevention mechanisms

information, software, services, as well as personnel and its qualification.

The risk for an asset, in the area of information security, is mostly defined by the three dimensions *criticality*, *vulnerability* and *threat*, as described in [4]. The criticality describes the importance of an asset for the organization and the impact a successful attack has. Vulnerabilities are properties of an asset that can be exploited, for example a bug or misconfiguration. A threat is able to exploit such a vulnerability. Risk management aims for the reduction of the overall risk level of an organization. In order to achieve this target, those three asset dimensions need to be reduced by means of detective, preventive and corrective nature.

Defense in depth, in information security, describes a basic strategy in decreasing these dimensions and tries to provide a cost-effective solution for reaching a desired risk level. It is mostly characterized as a layered approach. This layering is a response to the realization that there is a certainty “that any one point of protection may, and probably will, be defeated” [15]. This implies that any security architecture concentrating on a single defense mechanism also inhibits a single point of failure. The layered approach of DiD is countering this by demanding multiple lines of defense, making it harder for an attacker to reach the intended target.

Figure 3 shows an example DiD strategy using multiple detection and prevention mechanisms. In the example, virtual private networking (VPN) is used for secure remote access. On the basis of rule sets, firewalls in combination with segmented networks are able to limit the ability for an attacker to move through the network. Intrusion detection systems make it possible to detect possible attacks and act accordingly. The protection is further improved by employing access control, where users and services need to be authenticated in order to perform actions on the systems. A strategy including a demilitarized zone (DMZ) concept decreases the possibilities for an attacker to spread through the network. A DMZ forms an additional network segment used, for example, to

exchange data between different network zones not connected directly.

For a DiD strategy to be effective, the different layers need to employ different technical protection mechanisms and be orthogonal to ensure a breach of one layer does not imply the breach of all other layers. By employing a sound DiD strategy, the chances for a successful exploitation by an attacker can be lowered even when the attacker is using attack paths that were not considered when implementing the security systems.

2.3 Security approaches in automation protocols

In traditional IT, secure communication protocols make use of cryptographic methods in order to meet the protection objectives. The signing and encrypting of messages, for example, can ensure their confidentiality and integrity.

A protocol increasingly used in the industrial automation domain is OPC Unified Architecture (OPC UA). It is designed as a machine-to-machine communication protocol for industrial automation systems based on the Transmission Control Protocol (TCP). It applies a server client architecture and has an inherent security model. A recent examination of the protocol by the German Federal Office for Information Security (BSI) [7] proved that the design of OPC UA is secure providing that the optional security mechanisms have been enabled.

A current approach is OPC UA TSN. Using techniques of Time Sensitive Networking (TSN), the OPC foundation and different manufacturers of automation components try to replace traditional industrial automation protocols by OPC UA. OPC UA TSN will guarantee jitter and real-time cycle times up to 2 ms. Provided that the security features will be enabled, OPC UA TSN might evolve security in the automation domain. Nevertheless, it is not standardized yet, and since automation devices tend to be used for decades, it will not replace traditional industrial automation protocols in the near future.

In contrast, traditional industrial automation protocols focus primarily on availability and real-time guarantees measured in milliseconds. Nevertheless, recommendations on the enhancement of PNIO in terms of security have already been given by [3]. PNIO is recommended to be extended by the introduction of security modules based on the concept of the safety profile in *PROFIsafe*. This extension should be comprised of an additional layer ensuring authenticity, integrity and confidentiality for PNIO process data. Currently, no such extension was introduced in the PNIO standard.

2.4 Related work

A broad overview of the state of the art in industrial control system security has already been given [17]. This work

promotes the idea of using industrial control system (ICS) testbeds for helping to identify, exploiting and mitigating cybersecurity vulnerabilities. It follows a security assessment approach to analyze the ICS architecture and identify vulnerabilities. The authors identify the attacks on controllers and sensors as two attack classes in the current ICS threat landscape and the false data injection (FDI) as one of the emerging threats in this domain.

In [22], we acted on that suggestion to setup testbeds for detecting vulnerabilities. We introduced *ISuTest*, a fully-automated security testing framework for industrial automation systems. This framework mainly focuses on detecting vulnerabilities in the implementation and configuration of systems whereas replay attacks exploit a design vulnerability of PNIO.

A compact overview of the system architecture of PNIO and its functionality can be found in [23]. More details about the alarm management in PNIO networks are being described in [9]. However, this work focuses mainly on the performance aspects of a PNIO system.

Replay attacks in the control system domain have been studied in [18]. Here, the authors are assuming an attacker trying to disrupt the operation of a control system by injecting control inputs without being detected. The main contribution of this work is a new problem formulation providing information on the feasibility of replay attacks on control systems.

In [5], the vulnerability of PNIO to Man-in-the-middle (MitM) attacks, when using PNIO conformance class A, has already been addressed. For this conformance class, classical network infrastructure components, like standard Ethernet switches, are allowed and do not need to be certified for the usage in a PNIO network. In this work, the authors describe the communication setup process and possible errors that can occur during the setup, like the duplicate assignment of station names or IP addresses. By using open-source tools like *ettercap*, the authors tried a MitM approach that was not successful because of the PNIO cycle speeds, as suggested.

The security of PNIO was also explored in [2]. Here, the authors followed two attack scenarios. In the first scenario, the systems are using a shared medium for their communication, while in the second one, the systems are communicating via a packet switched network. In the shared media scenario, the attack method is to inject PNIO frames so that they will reach the target before the correct frame. This is requiring highly accurate timing and sound understanding of the protocol. In the packet switched scenario, a MitM attack is used to intercept and alter PNIO frames. This is a far more simple approach than in the first scenario, as the attacker only has to wait for frames to arrive and forward them after they are altered without the need to know any details about the AR between PNIO controller and device. While being successful in implementing the attack, the author noticed a certain

unreliability due to the fact that their PC-based attack system is partially delaying the frame processing.

Two attack types, the denial-of-service (DoS) and the MitM attack, and corresponding attacks on PNIO have also been studied by [19]. The attack on the DCP protocol relies on fabricated *DCP Ident Requests* frames preventing the successful assignment of a device name. Similar to that, the IP address assignment can also be prevented by sending malicious DCP or ARP frames. Both of these attacks will result in a DoS. The presented MitM attacks are divided into attacks during the startup process and during the operational stage. The MitM attacks rely on the *port stealing* method which is manipulating the switch to be able to capture the traffic between PNIO controller and device. In this work, the authors also investigate the setup of an intrusion detection system for ICS based on anomaly detection and show that this IDS is able to detect the described DoS and MitM attacks. The proposed anomaly detection is thereby only using the sequence of DCP frames to determine abnormal behavior, without incorporating indicators from the PNIO protocol or using means to detect the port stealing itself. In section 6.3, we will use this work for a comparative evaluation of our proposed solution.

There have been further efforts to study the security of PNIO. The security of *PROFIsafe* (an additional layer on top of PNIO dealing with safety aspects) has been examined in [1].

3 Attack case study

The attack case study section is first introducing the attacker model used in this work. Afterwards, the case study environment setup is depicted and the selected attack types are described. The section finishes with details on the attack detection techniques employed and their implementation in the evaluation setup.

3.1 Attacker model and attack origins

In this work, we assume an outside attacker. Without any knowledge of the process being performed, he aims at disrupting the industrial process, taking into account permanent damage to the system.

We distinguish three different origins within the production network from which the attacker launches his attacks:

ORIGIN_{Remote} is at the Remote HMI (human machine interface). The attacker starts with gaining access to the enterprise network. The target is for example a manager which has the permission to access the Remote HMI. An example for this attack would be sending a crafted e-mail to the manager. When opened, malware attached to the e-mail will be executed and attack the system.

ORIGIN_{OPCUA} is the OPC UA server. The server has been infiltrated by the attacker, for example by exploiting additional vulnerabilities from the Remote HMI as attacked in *ORIGIN_{Remote}*.

ORIGIN_{Local} is the Local HMI, a device which is attached directly to the same switch as the PLC. The attacker has gained access to this device, for instance by starting from *ORIGIN_{OPCUA}* and exploiting permissions and/or vulnerabilities.

3.2 Environment setup

The scenario for this work is implemented in an industrial IT security laboratory. This lab consists of both a physical part and a virtualization environment as described in [20]. Within the physical part, real industrial components like PLCs, sensors and actors, switches etc. are connected to each other using typical industrial protocols like PNIO, Siemens S7, EtherCAT and OPC UA.

In the virtualization environment (in the following referenced as “cloud”), different networks have been set up using virtual switches as well as virtual firewalls. Virtual machines connected to these networks provide components of automation systems (e.g. PLC programming stations, SCADA servers), management tools as well as different attack and detection tools.

The scenario, as depicted in Figure 4, makes use of a Siemens PLC (S7-300), a stepping motor (Schneider Electric mdrive), an HMI (Siemens KTP-700) and an industrial switch (Hirschmann RS-20) in the physical part. Additionally, there is a sniffer machine attached to the switch on a port which receives all traffic provided by the former three devices (port mirroring).

The physical process implemented in the scenario turns a disk, which is mounted on the motor shaft, clockwise from one position to the next with a pause of 2 seconds. The real industrial process from which our demonstration setup was derived, consists additionally of a driller which drills a hole in a workpiece if and only if a workpiece was transported by the disk to the drilling position.²

The physical switch is connected to the cloud via a network trunk. In the virtual part of this scenario, there is an OPC UA server (Siemens OPC gateway), a remote HMI (OPC UA based visualization) and a programming station (Siemens TIA portal) attached to the production network as well as a sniffer machine.

Both mirror streams from the physical and the virtual part of the production network are directed to a concentrator which merges them and provides the whole traffic of the production network to the detection tools.

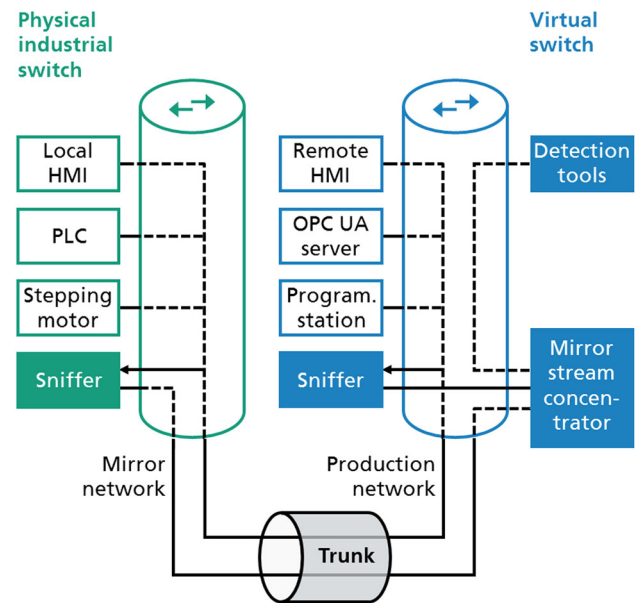


Fig. 4 The base scenario is set up using a PLC with a HMI and a motor in a star topology with a switch that connects the local network to a cloud network which hosts both virtual automation systems and the detection tools

3.3 Attack selection and execution

In order to steer the stepping motor, we focus on replaying network traffic. This is a very general attack which can be even performed by an outside attacker which is assumed for this work. There is no need to have knowledge about the process that is implemented, only the ability and time for watching and recording the traffic that is flowing through the network is needed. A limitation of that approach is that you can only record actions that occur while you are watching the traffic. But we don't consider this as a major drawback because most industrial processes rely on a very correct timing,³ so you can harm the system even by triggering valid actions in an invalid time slot. For instance, consider the drilling example from above with a driller drilling a hole into the disk instead of the workpiece.

3.3.1 Taking over control

Replaying traffic in order to attack a specific PNIO device requires the attacker to take over control of at least one connection to it. For that, we selected two types of attacks which turned out to be suitable. Despite the focusing on two replay attacks, the detection of other types of replay attacks can be transferred: By definition, each replay attack interrupts an existing communication relation because typical PNIO devices can maintain only one single PNIO connection.

² For obvious reasons, attacks causing irreversible damages and possibly injuring people are not allowed to be performed in our laboratory.

³ PNIO supports up to 5 ms cycle time (1 ms in the isochronous real-time class).

The first attack is a classical way to attack Ethernet-based protocols: Using *port stealing*. Usually, a network switch learns which device is connected to which hardware port by maintaining a lookup table. If it receives an Ethernet packet with the source MAC address M_i from its port P_j , it learns that packets with the destination MAC address M_i have to be delivered to port P_j . Former entries with the same MAC address get lost. This standard mechanism is useful because it adapts itself when new devices are added to the switch or when the topology changes.

The port stealing attack exploits this behaviour as follows: The device to be attacked has the MAC address M_{victim} and is attached to port P_{victim} . The attacker is attached to port $P_{attacker}$ and sends any kind of Ethernet-based packets with the source MAC address M_{victim} . The switch now learns that packets for M_{victim} have to be delivered to $P_{attacker}$. If the victim sends itself a network packet, the switch will recover the original relation of M_{victim} on port P_{victim} , but the attacker sends its port stealing packets in such a high frequency that these changes won't last for a longer period of time.

In our work, the PLC acted as the victim for the port stealing attack. Figure 5 illustrates the attack sequence with the corresponding network communication. When the motor sends packets to the PLC, they will not arrive at the PLC anymore but at the switch port of the attacker. Meanwhile, the attacker is able to send packets directly to the motor. The real PLC would be able to send packets to the motor as well, but that does not pose as a problem for the attacker, because the PNIO standard specifies, that if there is no real-time response from the communication partner, an existing connection has to be closed.

The second attack type is specific for PNIO operated devices: A *reconfiguration attack* using DCP. Caused by the design of this protocol without any authentication measures, each device that is located in the same subnet is able to reconfigure a PNIO device. There are two options available: *Set IP* and *Set Name*. Since the identifier of a PNIO device is its name, we chose that option for this attack.

In Figure 6, the sequence diagram of the DCP reconfiguration attack is shown. There are three devices involved with both their IP and MAC interface: The PLC, the motor with its original device name D_{old} and the attacker. Before the attack happens, PLC and motor exchange real-time data (RT).

The attacker asks the motor to change its name to D_{new} by sending a *DCP Set Request*. The motor emits a PNIO alarm frame "device name changed" to its Profinet communication partner, the PLC. The PLC acknowledges the alarm frame, and the connection between the PLC and the motor will be terminated. As a last step, the motor acknowledges the *DCP Set Request* by sending a *DCP Set Response* to the attacker. On a regular basis, the PLC keeps on asking for a device with the name D_{old} which is still configured in its setup, but will

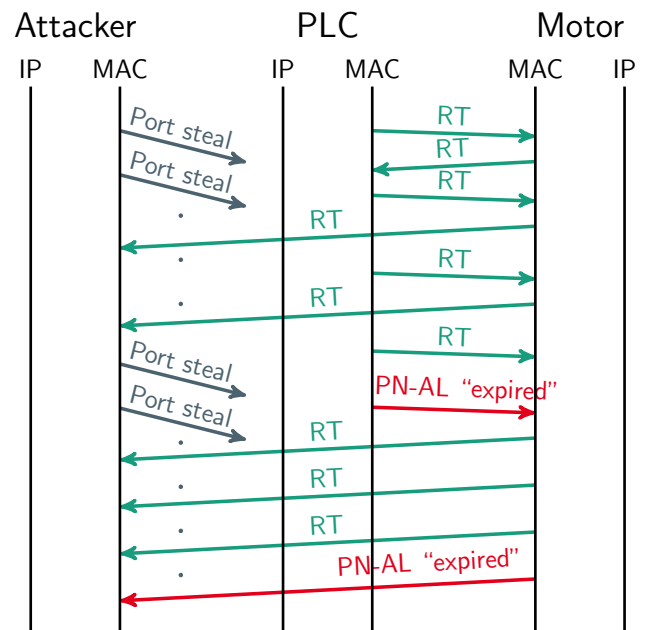


Fig. 5 The attacker performs a port stealing attack targeting the PLC via the hardware switch. This results in the attacker taking over the RT connection to the motor from the PLC

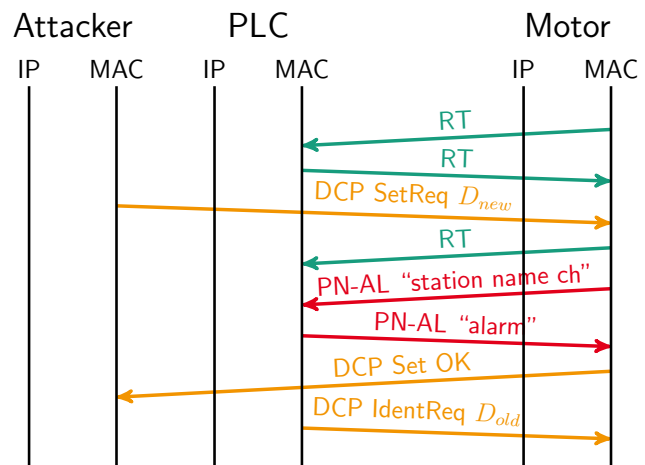


Fig. 6 The attacker reconfigures the motor using DCP. In consequence, the real-time communication between the PLC and the motor will be terminated

not receive any response, as no device is configured to react on that name now.

3.3.2 Replaying traffic

Replaying traffic successfully in a PNIO operated system requires, besides a communication connection, the application relation (AR) as mentioned before (section 2.1.2). Once a PNIO device like the stepping motor has established an application relation with its PLC, it refuses each further communication request, responding with an error reply. Hence,

the existing communication relation between the PLC and the motor has to be interrupted before continuing the replay.

Fortunately, both types of attacks described in section 3.3.1 ensure that the original communication relation gets interrupted: In the port stealing case, the PLC does not receive real-time data from the motor anymore, because this data is now redirected to the attacker. After the *DataHoldTime*, as described in section 2.1.2, has passed without any message from the motor, it terminates the connection and sends an alarm frame. As a consequence, the motor will also stop receiving real-time data from the PLC and will terminate its communication in the same way. In the case of the DCP reconfiguration attack, the connections gets lost immediately.

Once the motor is not involved in a communication relation anymore, the attacker is able to setup a new connection with the motor. The attacker now starts with *CM Connect Request* that is answered by the motor with a *CM Connect Response*. After that, the attacker sends a *CM Control Request*, which is answered with a *CM Control Response* by the motor. In the original traffic, the motor proceeds by sending a *CM Control Request*, but the attacker does not have to reply to that request and can directly start replaying the PNIO real-time packets. As a side note, the attacker does not even need to process or react on the replies of the motor.

4 Attack detection

In this section, we will start with a model for detecting the attacks being described in the previous sections. In the second part, we explain how we implemented this using our existing open-source based intrusion detection framework.

4.1 Model for attack detection

In the case of the port stealing based replay attack, there is the evidence of the packets used for the port stealing. So in general, one could be suspicious if such unusual traffic can be observed in the network. Since the port stealing approach is based on the attacker sending forged packets in a limited period of time⁴ between a correct packet from the device under attack and the response to this packet, the attacker has to produce lots of packets in time. A method to detect this huge amount of traffic is by using flow-based intrusion detection systems. Those systems aggregate single packets by considering source and destination as well as some other configurable keys to flows of packets. Another method for detection would be a switch configuration that emits alerts when MAC addresses change from one interface to another. We will call that evidence $\langle \textit{port-stealing} \rangle$ and the tool performing that analysis *Port stealing detector*.

⁴ In tests, we discovered answer times less than 1 ms.

Additionally, there will appear two PNIO alarm frames of the type “*AR consumer DHT/WDT expired (RTA_ERR_ABORT)*” at the network: The first one from the PLC that does not receive cyclic data from the motor anymore ($\langle \textit{al-plc-expired} \rangle$), and a second one from the motor complaining about the PLC stopping to send cyclic data ($\langle \textit{al-motor-expired} \rangle$).

Detecting the DCP reconfiguration attack is very different because there will not be any packets present that are unusual for the specific networks. Nevertheless, industrial networks tend to be very stable and are run for a long time. So reconfiguring devices is not part of the daily work and for that reason, detecting DCP packets that change the name of a device seems to be reasonable.⁵ We will call that packet $\langle \textit{dcp-setname} \rangle$.

In order to refine the detection, a PNIO alarm frame “*station name changed*” will appear on the network if the exchange of name was successfully completed ($\langle \textit{al-station-name-changed} \rangle$). This alarm gets acknowledged by the PLC with an alarm ($\langle \textit{al-generic} \rangle$). And further on, the PLC will send regularly *DCP Ident Requests* asking for the initial name of the motor ($\langle \textit{dcp-ident-req-oldname} \rangle$).

Summarizing, the port stealing attack can be discovered using the following evidence:

$\langle \textit{port-stealing} \rangle \text{ AND } \langle \textit{al-plc-expired} \rangle \text{ AND } \langle \textit{al-motor-expired} \rangle$

In case of the DCP reconfiguration attack, we experience the following evidence:

$\langle \textit{dcp-setname} \rangle \text{ AND } \langle \textit{al-station-name-changed} \rangle \text{ AND } \langle \textit{al-generic} \rangle \text{ AND } \langle \textit{dcp-ident-req-oldname} \rangle \text{ (repeated)}$

It has to be noted that not each evidence of the AND clauses is really needed for assuming that there is an attack ongoing: In the first case, it would be sufficient to discover $\langle \textit{port-stealing} \rangle$ and one of $\langle \textit{al-plc-expired} \rangle$ or $\langle \textit{al-motor-expired} \rangle$, because it is possible to infer that when one partner of the communication quits, the other one will follow soon. In the second case, $\langle \textit{al-generic} \rangle$ and $\langle \textit{dcp-ident-req-oldname} \rangle$ only improve the accuracy of the detection. This is an important notice because in some topological constellations, not every packet is visible for the IDS (see section 6).

⁵ In order to eliminate false positives, we expect the operator receiving alerts from the IDS will be aware of any maintenance work.

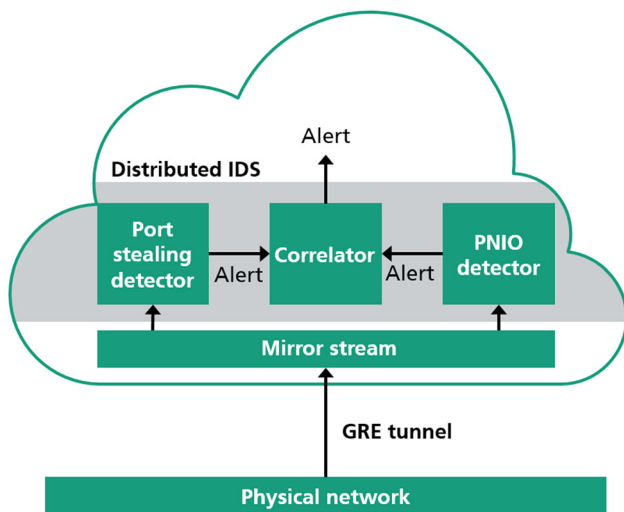


Fig. 7 Schema of the implementation, comprising attack detection tools in the cloud being fed by a mirror stream from the physical part of the network where the attacks take place

4.2 Attack detection implementation

Figure 7 depicts a schema of the attack detection systems. The mirroring of the network traffic in the physical environment is ensured by the hardware switch which monitors the specific ports of the PLC and the motor. Since the distributed IDS resides in the virtual part of the laboratory, the mirrored traffic has to be forwarded using a generic routing encapsulation (GRE) tunnel into the cloud.

There are two consumers of the network packets from the *mirror stream*: The *Port stealing detector* and the *PNIO detector*. Both create *Alerts* if they detect any suspicious activity in the network packet flow.

The *Correlator* receives those alerts. It is able to aggregate them and generate new alerts which can be sent to the operator.

4.2.1 Port stealing detector

On the one hand, the *Port stealing detector* sniffs the packets from the mirror stream. The open IPFIX standard (Internet Protocol Flow Information Export, [8]) specifies flow keys for layer 2 communication like MAC source and destination addresses. Unfortunately, current implementations of the common open-source flow-based IDS *Flow-tools* [10], *Silk* [16] and *nfdump* [11] do not support those keys, as they are limited to layer 3 communication. For that reason, we are currently not able to use a flow-based IDS as a port stealing detector.

As an alternative solution for implementing the port stealing detector, we have set up the switch connecting the physical components to send alerts to the correlator when MAC addresses change from one interface to another. The

```

1 alert (msg: "Controller expired Alarm";
2   profinet: alarm io-device -> io-controller
3   [ErrorCode1 == 253, ErrorCode2 == 5];
4   gid: 8892; sid: 9088941; rev: 1;)

```

Listing 1 Example of a Snort PNIO pre-processor rule detecting a PNIO expired alarm.

great advantage of this solution is that it is available. The drawback is the fact that it needs reconfiguration of the switches which is not feasible in every production environment.

4.2.2 PNIO detector

On the other hand, the *PNIO detector* receives the network packets from the mirror stream. In a first iteration, it consisted of a Python script, running in background and employing the “Network packet and pcap file crafting/sniffing/manipulation/visualization security tool” Scapy [6]. It filtered the Ethertype *0x8892* of PNIO and generated an alert when it received either a specific PNIO alarm frame (see section 2.1.4) or a DCP packet (see section 2.1.1). The reason for not using a standard network-based IDS like Snort [24] was that those tools are based on IP traffic, while PNIO is based on Ethernet.

In the meantime, we developed a so-called pre-processor for Snort. This pre-processor enables the Snort IDS to react on PNIO Ethernet frames. The reaction is triggered by specific rules which have been defined according to the former Python script. The advantage of the pre-processor is that it integrates in a standard IDS. This includes the use of well-developed tools and techniques to handle high loads of network traffic and alerts.

Listing 1 depicts a PNIO pre-processor rule which gets triggered when the respective PNIO alarm frame passes the mirror stream. Line 1 defines an alert and assigns it an alert message. Line 2 refers to the PNIO pre-processor using the keyword *profinet* which is registered for the Ethertype *0x8892*. Additionally, it specifies that the network packet must be an alarm sent from a PNIO device to a PNIO controller. Line 3 detects whether the error codes specified for an PNIO expired alarm are set. Line 4 closes the alert with organizational information.

In listing 2, a resulting Snort alert is depicted. Line 1 starts with the identifiers of the rule and the alert message. Line 2 repeats the textual information in error codes. Line 3 and 4 as well as line 5 and 6 provide information about the sender respective the receiver of the PNIO alarm: MAC address, PNIO role and PNIO device name.

This is just a quick overview of the developed PNIO pre-processor for Snort applied to the attack case being used in

```

1  [**] [8892:9088941:1] Controller expired Alarm
2  [ErrorCode2 == 5; ErrorCode1 == 253;] [] {ALARM}
3  40:D8:55:XX:XX:6C (IO-Device, "mdrive",
4  192.168.X.22) ->
5  28:63:36:XX:XX:44 (IO-Controller, "plcmaster",
6  192.168.X.20) [**]

```

Listing 2 The resulting Snort alert for a PNIO expired alarm.

this work. A more complete presentation will be performed in future work.

5 Attack prevention

In this section, we examine how to prevent the attacks described in section 3 with means of defense in depth. As motivated in section 2.2, it is important to apply different, orthogonal protection mechanisms. Nevertheless, we focus on the segmentation of networks using firewalls with flow control and the usage of a DMZ. These are the most appropriate countermeasures for network attacks like the replay attacks presented in this work.

In Figure 8, four different prevention scenarios are depicted. Cylinders represent network switches, each text row inside them represents a server connected to the specific switch. Firewalls with flow control are depicted as walls decorated with flames. The firewalls separate the network in different zones which are colored and named accordingly. It has to be noted that this is not a complete picture of an automation enterprise network, only assets relevant to the scenarios are depicted in the figure. As a general assumption, we depict a firewall in between the enterprise and the production network.

PREV_{None} illustrates the base scenario without any segmentation in the production network, as introduced in section 3.2. As an example for a system in the enterprise network, a manager PC (*Mgr PC*) is shown. Inside the production network, there are two different switches: On the left side, a virtual switch is providing network access to the remote HMI (*R-HMI*), the OPC UA server (*OPC UA*) and the programming station (*Pr. stat.*). On the right side, a physical switch connects the physical field devices local HMI (*L-HMI*), the PLC and the stepping motor (*Motor*).

PREV_{DMZ} makes use of a DMZ inside the virtual network. Both the OPC UA server and the programming station are located inside the DMZ, whereas the remote HMI resides in the enterprise network. Connections from the enterprise to the production network terminate inside the DMZ, direct access to the physical devices is not permitted. The firewalls establishing the DMZ may be either virtual or physical ones.

PREV_{ProdFw} puts a firewall in front of the PLC separating the PLC and the stepping motor from the rest of the physical

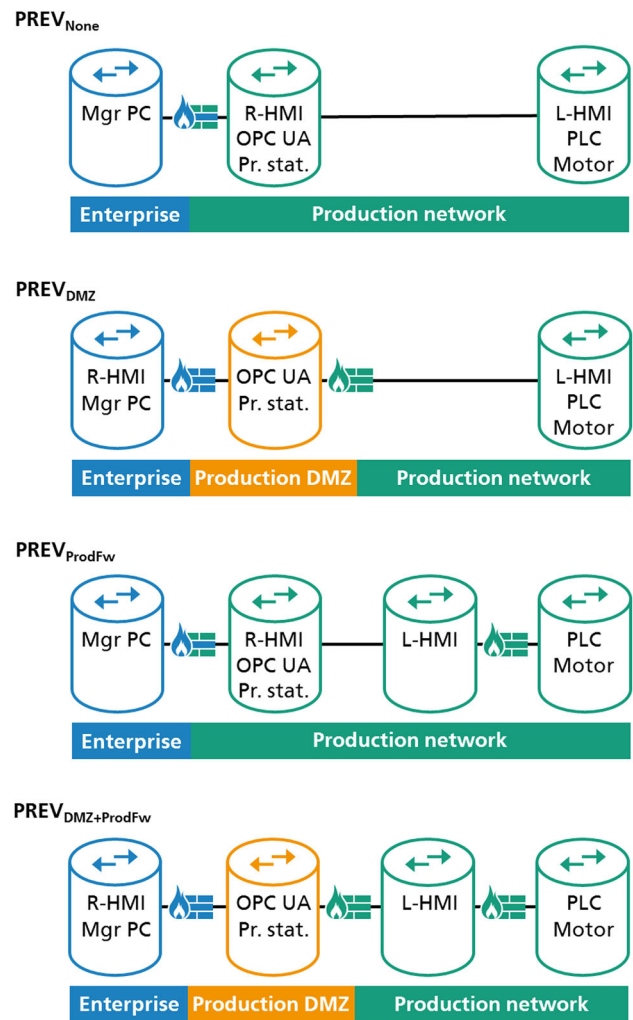


Fig. 8 Prevention scenarios: PREV_{None} is the base scenario without any firewalls applied, whereas PREV_{DMZ} adds a DMZ, PREV_{ProdFw} a local firewall in front of the PLC, and PREV_{DMZ+ProdFw} combines both

network. Because of the real-time communication between the PLC and the stepping motor, a scenario with a firewall in front of both physical asset cannot be deployed: The packet inspection process of the firewall would not be fast enough. The virtual part of both the enterprise and the production networks are the same as in the base scenario PREV_{None}.

PREV_{DMZ+ProdFw} is a combination of PREV_{DMZ} and PREV_{ProdFw}: The DMZ approach is applied as well as the local firewall in front of both the PLC and the stepping motor.

5.1 Flow control

Deploying firewalls is only the first step of securing a network. In order to control network flows, it is necessary to add specific rules for network traffic defining what is allowed and what is forbidden.

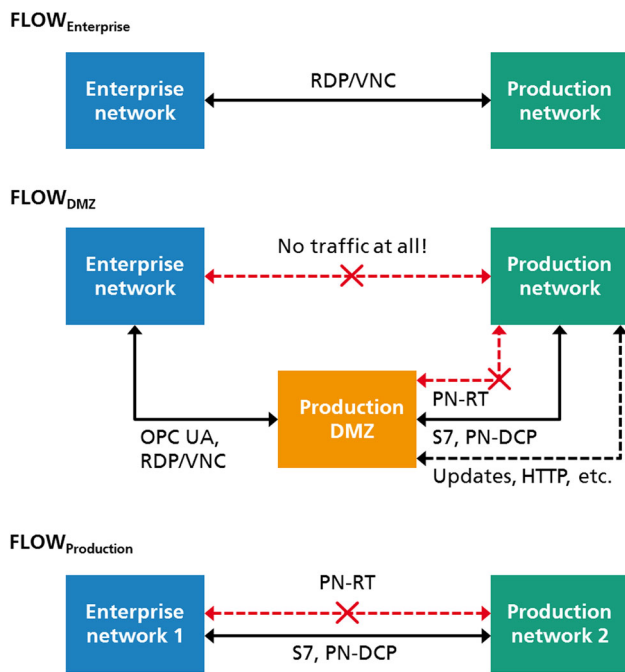


Fig. 9 Allowed and forbidden traffic flows in three different firewall scenarios

In Figure 9, the different networks of the scenarios are depicted, including allowed and forbidden network traffic flows. It should be noted that the default rule is a “deny all”, which allows only traffic explicitly described to pass.

FLOW_{Enterprise} applies to the firewall between the enterprise and the production network as used in the scenarios $PREV_{None}$ and $PREV_{ProdFw}$. In order to access the remote HMI and the programming station, a desktop sharing protocol like RDP (Remote Desktop for Windows) or VNC (Virtual Network Computing) needs to pass the firewall. There is no need for any automation traffic, like PNIO, to pass the firewall.

FLOW_{DMZ} applies to the scenarios $PREV_{DMZ}$ and $PREV_{DMZ+ProdFw}$ which implement a production DMZ. As a basic characteristic, direct traffic from the enterprise network to the production network is not possible. Each traffic flow from the enterprise as well as from the production network terminates in the DMZ. In contrast to $FLOW_{Enterprise}$, OPC UA has to pass from and to the enterprise network in order to operate the R-HMI.

Between the DMZ and the production network, S7 and DCP has to be allowed. Both protocols are used by the Siemens TIA portal to configure, maintain and debug Siemens PLCs and PNIO devices. Some PNIO devices even offer a web interface. Depending on the business needs, it has to be decided whether access via HTTP or HTTPS should be allowed.

It also needs to be decided how to update servers in the production network, which is crucial in order to patch

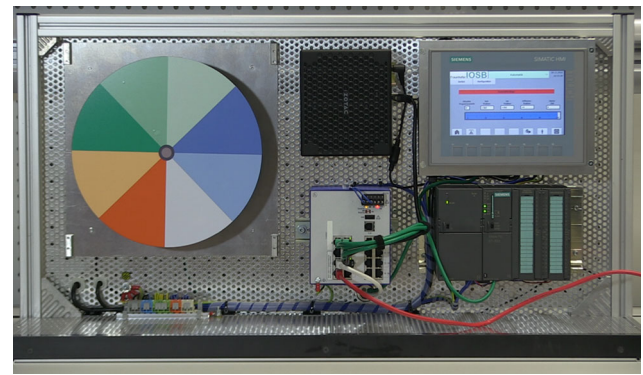


Fig. 10 The demonstrator with a stepping motor turning a disk used for performing the replay attacks

vulnerabilities on a regular basis. A basic scenario would allow a server in the production network to connect to an update server in the enterprise network. A more advanced scenario would implement a local update server in the DMZ which gets new updates pushed from the enterprise update server. Nevertheless, these considerations are not part of this research.

Explicitly not permitted is real-time traffic from the DMZ to the production network.

FLOW_{Production} applies to the scenarios $PREV_{ProdFw}$ and $PREV_{DMZ+ProdFw}$ which separate the production network in two different subnets. S7 and DCP have to be allowed to pass because of the programming station, while RT must not pass.

6 Evaluation

In this section, we will evaluate our approach to detect and prevent the two replay attacks dedicated to PNIO networks described in section 3.3. For the detection, we performed and recorded several tests in different setups. Whereas the prevention evaluation is a conceptual one based on both the theory of firewalling and our observations and experiences within our laboratory. The section concludes with a comparative evaluation of the presented detection solution.

6.1 Detection evaluation

Figure 10 depicts the base setup with the PLC (lower right) and the motor (hidden behind the colored disk on the left) in a star topology connected by a switch (lower middle). In the upper middle resides an embedded PC which is connected both to the mirror port of the switch and a cloud uplink. An HMI, reflecting the status of the system, is placed in the upper right.

Table 1 enumerates the scenarios considered in our evaluation. The first column represents the identifier of the

Table 1 The different setups examined in the evaluation of the replay attack detection

#	Topology	Attack origin	Action	Evidence	Detection	Remarks
1	Star	L-HMI	Port stealing	PS, P_{ex}, M_{ex}	Yes	
2	Star	L-HMI	DCP reconfiguration	$SET, M_{snc}, P_{gen}, ID$	Yes	
3	Star	R-HMI, OPC UA	Port stealing	PS, P_{ex}, M_{ex}	Yes	
4	Star	R-HMI, OPC UA	DCP reconfiguration	$SET, M_{snc}, P_{gen}, ID$	Yes	
5	Line	L-HMI	Port stealing	PS	No	False negative
6	Line	L-HMI	DCP reconfiguration	SET, ID	Yes	
7	Line	R-HMI, OPC UA	Port stealing	PS	No	False negative*
8	Line	R-HMI, OPC UA	DCP reconfiguration	SET, ID	Yes	
9	Star	–	PLC plug removed	M_{ex}	No	
10	Star	–	Motor plug removed	P_{ex}	No	
11	Line	–	PLC plug removed	–	No	Motor keeps spinning
12	Line	–	Motor plug removed	–	No	Motor stops
13	Star	R-HMI, OPC UA	PLC change name	–	No	Siemens S7 traffic
14	Star	R-HMI, OPC UA	Motor change name	–	No	Proprietary protocol traffic

* In the previous paper [21], we evaluated this case as “unsuccessful attack”. However, in further testing, we came to the conclusion that the attack also succeeds like in case #5

scenario. The second column describes the topology of the demonstrator: In case of the “star” topology, both PLC and motor are connected to the switch. In the “line” case, the motor is attached directly to the second Ethernet port of the PLC.

On the one hand, the attacks can be performed locally ($ORIGIN_{Local}$) by using a malicious laptop, or by infecting a physical device like the L-HMI. In both cases, the attacker is connected to the switch on a port that is configured with the same VLAN as the PLC and the motor. On the other hand, the attacks can be launched from a remote location (OPC UA, R-HMI). Since the attack detection evaluation does not take any preventive methods in account, there is no distinction between $ORIGIN_{OPCUA}$ and $ORIGIN_{Remote}$. The origin is being represented by the column “Attack Origin”.

The column “Action” tells which type of action has been performed in the specific scenario: Mainly the two types of attacks, but additionally, two cases have been considered in which we expected PNIO alarm frames similar to the ones being observed in the attack case. On the one hand, this is the removal of either the plug of the PLC or the plug of the motor from the switch. On the other hand, this is the substitution of the PNIO name by the respective programming station: In case of the PLC the *Siemens TIA portal*, in case of the motor, the *Lexium Software Suite*.

In the column “Evidence”, we have listed abbreviations of the evidence being described in section 4.1: PS stands for $\langle port-stealing \rangle$. The different PNIO alarm frames are prefixed with their origin: P stands for an alarm that has been issued by the PLC, M for one that has been sent by the motor. The index denotes the type of alarm: ex for

$\langle al-\frac{plc}{motor}-expired \rangle$, snc for $\langle al-station-name-changed \rangle$ and gen for $\langle al-generic \rangle$. There are also abbreviations for several DCP packets: SET stands for $\langle dcp-setname \rangle$, while ID stands for $\langle dcp-ident-req-oldname \rangle$.

The next column “Detection” bares witness of the fact whether our proposed IDS detected the attack or not.

In our evaluation, we considered 14 different scenarios. The first eight cases arise from the topology permutation of the demonstrator, the origin and the type of the attack. Case #9 up to #12 reflect the permutation of the location where the cable has been unplugged and the topology, while the last two cases describe the interaction of both the PLC and the motor with their respective management stations located only in the virtual part of the production network.

First of all, it has to be noted that both types of replay attacks succeed in the case of a star topology. However, if the physical components are connected in a line topology, replaying does not succeed completely using the port stealing attack, but it still does when employing the DCP reconfiguration attack. The partial success of the replay attack in a line topology expresses in such a way that depending on random packet timings⁶, the motor alternates in executing the commands of the real PLC and the attacker. Nevertheless, this is also a serious attack since the motor does not act as intended and may cause harm.

The proposed detection works in case of an attacker that sends the replay packets from a remote location. In case of a line topology, the port stealing attack will not be detected resulting in two *false negatives* (cases #5 and #7). The reason for not detecting the attack in that case is that PNIO alarms do

⁶ caused by the “best effort” delivery approach in networking devices

Table 2 Prevention scenario analysis regarding attack origins 1 to 3

	Prevention scenarios			
	PREV _{None}	PREV _{DMZ}	PREV _{ProdFw}	PREV _{DMZ+ProdFw}
ORIGIN _{Remote} (R-HMI)	PS: X DCP: X	PS: ✓ DCP: ✓	PS: X* DCP: X**	PS: ✓ DCP: ✓
ORIGIN _{OPCUA} (OPC UA)	PS: X DCP: X	PS: X* DCP: X**	PS: X* DCP: X**	PS: X* DCP: X**
ORIGIN _{Local} (L-HMI)	PS: X DCP: X	PS: X DCP: X	PS: X* DCP: X**	PS: X* DCP: X**

✓ = attack prevented
 X = attack not prevented
 X* = Attack impact reduced to temporary denial-of-service attack
 X** = Attack impact reduced to persisting denial-of-service attack

not protrude the connection between the PLC and the motor. To overcome this issue, it is necessary to place a network tap in between both physical components and forward these packets also to the mirroring stream.

In the case of the DCP reconfiguration attacks in the line topology (cases #6 and #8), only the DCP packets can be discovered. Nevertheless, it is not impossible to derive from the observed *DCP Set Request* and the subsequent repeated *DCP Ident Requests* that an attack has taken place.

In order to validate the proposed IDS, we undertook tests which might result in similar alarms: Removing a cable from the switch in the star topology case results in either an “expired” alarm from the motor or the PLC. Since there is neither a port stealing attack active nor any DCP packets can be discovered, the IDS does not generate an alert. That happens as well for the management station changing the names of the devices: Neither the *Siemens TIA portal* nor the *Lexium Software Suite* make use of DCP for that reason. Both use other protocols for that purpose: Either Siemens-S7 or a proprietary protocol. Nevertheless, the Siemens TIA portal makes use of DCP for other purposes like the discovery of PNIO devices. Summarizing, in none of the tested cases a *false positive* appeared.

Additionally, we evaluated the performance of the PNIO detector, the Snort pre-processor. As a test environment for performance evaluation, we used a physical machine (Intel i7, 2.6 GHz, 8 GB RAM) running *Ubuntu 16.04.1* and *Snort 2.9.8.3*. We replayed network packets using *tcpreplay 3.4.4* on a virtual network interface and checked the maximum load when Snort analyzes 100% of the network packets.

It turned out that Snort could process up to 143,000 PNIO packets per second (pps) with the enabled pre-processor (164 megabit per second (mbps)). With a reasonable PNIO alarm ratio of 0.03%, this performance did not change. Turning the alarm ratio to more than 90%, Snort could process only 32,000 pps (16.3 mbps).

In a typical PNIO setup, PNIO devices and controllers with a minimum cycle time of 1 ms transmit $2 \times 1,000$ pps.

Theoretically, this setup could therefore be used to monitor PNIO traffic for more than 100 PNIO devices.

6.2 Prevention evaluation

Table 2 shows the results of the attack prevention achieved by different DiD measures. In the rows, the three attack origins described in section 3.1 are depicted. In the columns, four scenarios, from the base scenario without any security controls up to the DMZ with an additional firewall, are shown.

In each table cell, there are results presented for both the port stealing and the DCP reconfiguration attack. A ✓ symbolizes that an attack has been prevented. In contrast, an X signalizes that the attack could not be prevented. There exist cases in which the prevention mechanisms lower the impact of the attack: In those cases denoted by asterisks, either a temporary or a persisting denial-of-service is being caused by the attacker, but no other malfunction of the stepping motor.

Without any prevention mechanisms applied, as indicated in the first column (base scenario PREV_{None}), each attack could take place unresistedly.

The DMZ approach PREV_{DMZ} helps, as expected, to prevent attacks from the ORIGIN_{Remote}. It also helps to lower the attack surface for attacks originating from inside the DMZ (ORIGIN_{OPCUA}). In these cases, the stepping motor is affected by a DoS attack. If caused by port stealing, the DoS ends when the attack ends, if caused by DCP reconfiguration, the DoS persists until the motor gets reconfigured manually. In case of an attack taking place locally, as from ORIGIN_{Local}, the DMZ is not able to improve the situation.

Prevention scenario PREV_{ProdFw}, using an additional firewall in the production network, helps to reduce the attack impact to a DoS. In contrast to PREV_{DMZ}, this is valid for all three attack origins.

The combination scenario PREV_{DMZ+ProdFw} combines the advantages of PREV_{DMZ} and PREV_{ProdFw}: From ORIGIN_{Remote}, the DMZ prevents the attack completely. Both the DMZ and the additional firewall lower the attack

surface to DoS for attacks originating from $\text{ORIGIN}_{\text{OPCUA}}$. From $\text{ORIGIN}_{\text{Local}}$, the additional firewall lowers the attack surface to DoS.

Concluding, complete replay attacks that steer the motor in a way not intended, can be prevented by applying both a DMZ and an additional firewall for all three attack origins. Nevertheless, DoS attacks may succeed stopping the stepping motor which may be a risk as well. From an overall prevention perspective, it is crucial to apply more layers of DiD mechanisms in order prevent attackers from getting inside the production network.

6.3 Comparative evaluation

The IDS proposed by [19] (in the following referred to as *PAUL*) also focuses on intrusion detection for PNIO networks. For that reason, we will try to perform a comparative evaluation with our approach. However, since there is not any implementation of *PAUL* available, we will have to restrict the comparison to the conceptual level.

PAUL derives n-grams of DCP packets from the mirrored network stream. Each n-gram that appears in a learning phase is considered a good one. Any other n-gram that can be observed in the regular operation phase is considered a bad one, depicting an attack.

In contrast to *PAUL*, we propose an IDS which does not need any learning phase but works out of the box once it has been configured for detecting security breaches in PNIO networks. Learning phases in industrial automation systems can be leveraged only in very rare cases since availability is the most important parameter. For example, in order to detect DCP packets which rename a PNIO device, you would have to interrupt existing communication relations and probably stop the industrial process.

As a second difference, our solution follows a dual approach combining both intrusion evidence from the well-known IT world with anomaly evidence from automation protocols. Whereas *PAUL* is limited to PNIO DCP, our IDS takes into account additional protocols of the PNIO protocol suite like CM and alarm frames. That renders our IDS to be more comprehensive than *PAUL*.

7 Conclusion and future work

In this work, we took a look at the security of the industrial Ethernet protocol *Profinet IO*, which is one of the most common automation protocols being installed in Europe. First, we explained relevant functions and mechanisms during the startup phase and while running. We also presented the concept of defense in depth and discussed the related work. In an attack case study, we introduced an attacker model with three different attack origins and the base scenario

we performed our research in. Afterwards, we motivated why we choose replay attacks as the means of attack and how the attacks were performed. We continued by presenting an attack detection model for an IDS that is able to detect the attacks, and by giving details on how this model was implemented in our scenario. In a next step, we introduced attack prevention following the defense in depth strategy. For this, the base scenario was extended with three additional ones using and combining different defense in depth mechanisms. We also described different flow control rules that can be used in firewalls to prevent replay attacks.

We then evaluated our detection approach which takes into account both alarms from the well-known IT world and different protocols of the Industrial Ethernet world. In 14 different test cases, we ended up with no false positive and two false negatives in a special constellation without any proper mirroring in place. We also performed a conceptual evaluation of our prevention approach with attack results for different scenario and attack origin constellations. This showed how the usage of a DMZ and network segmentation can improve attack prevention and how a layered defense in depth strategy can further improve the results. The best results can be achieved when different preventions mechanisms are in place. We concluded our evaluation with a comparison of our detection mechanism with another one and were able to show the advantages of our proposed solution.

As evaluated, it is possible to make the network more resistant to cyberattacks. Nevertheless, the deeper the attacker intrudes into the network, the more complex or even impossible is it to defend the process and keep it running undisturbedly. Taking care of replay attacks is very crucial since PNIO devices are a worthwhile target. This is the case because an attacker does not need any special skills or knowledge of the process being controlled, making these attacks easy to perform.

In order to enable data exchange with other systems in the use case of the so-called “Industrie 4.0” or “(Industrial) Internet of Things”, many companies connect their industrial networks with their office networks and the Internet. Improper and insecure configurations of these connections are raising risk and danger. And the danger is proverbial: Industrial processes control real devices. If e.g. motors, valves, robots do not behave as expected, serious physical damage to property and lives may happen. This strengthens the need for intrusion detection and prevention systems adapted for the industrial environment.

7.1 Future work

Implementation and testing was done in a prototypical scenario. We will investigate further and deploy different

firewalls of various vendors in the network. A challenge is the variety of possibilities to configure such industrial firewalls. Some of them even offer the feature to do a deep packet inspection which might help to prevent further attacks.

When deploying DiD mechanisms as presented in this work, the steering of the motor by the attacker could be prevented. Nevertheless, the attacks could take effect as DoS. In order to prevent even this DoS, a possibility is to use white-listing. White-listing in firewalls pins not only communication protocols or ports to specific roles, but specific hardware devices using their MAC address. However, this hardware address may be spoofed.

So one step further is to deploy white-listing to network switches. This kind of white-listing pins hardware addresses to switch ports. Assuming a remote attacker that is not able to access the switches physically, this makes PNIO systems more secure. As drawback, it lowers also the flexibility which is inherent to PNIO: Exchanging a defective PNIO device requires not only an automation specialist that is able to configure a PNIO device name, but also a network engineer that reconfigures the network infrastructure. Nevertheless, it is worth a try.

Performing replay attacks in automation systems is only one type of possible means to disturb or destroy an industrial processes, yet a simple and easy to perform one. There are many more attack vectors, like the tampering of data that is presented in visualizations, worth being considered and assessed. Evaluation criteria include the amount of knowledge about the industrial process an attacker needs, how easy the attacks are to employ and the extend of possible damage. This work will be continued by an overall attack case study of the used demonstrator including the PLC, the motor, the local HMI as well as an HMI that accesses data from the PLC and presents it on a web interface in the enterprise network.

Many automation systems in Europe are operated with PNIO. Nevertheless, there are many other automation protocols which are widespread. We will setup demonstrators with protocols besides PNIO (for example with EtherCAT) and will try to transfer our attack and detection techniques.

An ongoing task is the implementation work in our distributed open-source IDS. This task deals with the main problem that common IDS start with the IP layer. They do not support Ethernet-based protocols as they are usually used in industrial automation networks and are not able to pay attention to the real-time requirements of such networks. An effort to close this gap was the development of a pre-processor for the popular IDS Snort which enabled it to deal with PNIO traffic. This work has to be continued in defining rules for the detection of more than replay attacks.

Another effort is the work on different correlation techniques that have to be employed in a central correlation engine that controls the possible flood of different alerts from both the traditional IT and the automation domain.

References

- Åkerberg, J., Björkman, M.: Exploring network security in profisafe. In: International Conference on Computer Safety, Reliability, and Security, pp. 67–80. Springer (2009)
- Åkerberg, J., Björkman, M.: Exploring security in profinet io. In: Proceedings of the 2009 33rd Annual IEEE International Computer Software and Applications Conference, COMPSAC '09, vol. 01, pp. 406–412. IEEE Computer Society, Washington, DC (2009). <https://doi.org/10.1109/COMPSAC.2009.61>
- Åkerberg, J., Björkman, M.: Introducing security modules in profinet io. In: 2009 IEEE Conference on Emerging Technologies & Factory Automation, pp. 1–8. IEEE (2009)
- Bass, T., Robichaux, R.: Defense-in-depth revisited: qualitative risk analysis methodology for complex network-centric operations. In: 2001 MILCOM Proceedings Communications for Network-Centric Operations: Creating the Information Force (Cat. No. 01CH37277), vol. 1, pp. 64–70 (2001). <https://doi.org/10.1109/MILCOM.2001.985765>
- Baud, M., Felser, M.: Profinet io-device emulator based on the man-in-the-middle attack. In: 2006 IEEE Conference on Emerging Technologies and Factory Automation, pp. 437–440 (2006). <https://doi.org/10.1109/ETFA.2006.355228>
- Biondi, P.: Scapy documentation. <http://www.secdev.org/projects/scapy/doc/> (2010). Accessed 08 Dec 2016
- BSI: Opc ua security analysis (2016)
- Claise, B.: Specification of the IP flow information export (IPFIX) protocol for the exchange of IP traffic flow information. RFC 5101 (2015). <https://doi.org/10.17487/rfc5101>. <https://rfc-editor.org/rfc/rfc5101.txt>
- Ferrari, P., Flammini, A., Vitturi, S.: Performance analysis of profinet networks. *Comput. Stand. Interfaces* **28**(4), 369–385 (2006)
- Fullmer, M., Romig, S.: The osu flowtools package and cisco net-flow logs. In: Proceedings of the 2000 USENIX LISA Conference (2000)
- Haag, P.: Watch your flows with nfsen and nfdump. In: 50th RIPE Meeting (2005)
- HMS Industrial Networks: Feldbusse heute. <http://www.feldbusse.de/trends/status-feldbusse.shtml> (2016). Accessed 08 Dec 2016
- HMS Industrial Networks: Variantenvielfalt bei Kommunikationssystemen. <http://www.feldbusse.de/Trends/trends.shtml> (2016). Accessed 08 Dec 2016
- IEC: 61158-6-10 Industrial Communication Networks—Fieldbus Specifications—Part 6–10: Application Layer Protocol Specification—Type 10 Elements. Standard, International Electrotechnical Commission, Geneva (2007)
- IEC: 62443-1-1 Industrial Communication Networks—Network and System Security—Part 1–1: Terminology, Concepts and Models. International Electrotechnical Commission (IEC), Geneva (2009)
- McHugh, J.: Sets, bags, and rock and roll. In: European Symposium on Research in Computer Security, pp. 407–422. Springer (2004)
- McLaughlin, S., Konstantinou, C., Wang, X., Davi, L., Sadeghi, A.R., Maniatakos, M., Karri, R.: The cybersecurity landscape in industrial control systems. *Proc. IEEE* **104**(5), 1039–1057 (2016)

18. Mo, Y., Sinopoli, B.: Secure control against replay attacks. In: 47th Annual Allerton Conference on Communication, Control, and Computing, 2009. Allerton 2009, pp. 911–918. IEEE (2009)
19. Paul, A., Schuster, F., Knig, H.: Towards the protection of industrial control systems: conclusions of a vulnerability analysis of profinet IO. In: Proceedings of the 10th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, DIMVA'13, pp. 160–176. Springer, Berlin (2013). https://doi.org/10.1007/978-3-642-39235-1_10
20. Pfrang, S., Kippe, J., Meier, D., Haas, C.: Design and architecture of an industrial IT security lab. In: Testbeds and Research Infrastructures for the Development of Networks and Communities, pp. 114–123. Springer (2016)
21. Pfrang, S., Meier, D.: On the detection of replay attacks in industrial automation networks operated with profinet IO. In: Proceedings of the 3rd International Conference on Information Systems Security and Privacy, pp. 683–693 (2017). <https://doi.org/10.5220/0006288106830693>
22. Pfrang, S., Meier, D., Kautz, V.: Towards a modular security testing framework for industrial automation and control systems: Isutest. In: Proceedings of the 22nd IEEE International Conference on Emerging Technologies and Factory Automation, ETFA 2017 (2017)
23. Popp, M.: Industrial Communication with PROFINET. PROFIBUS Nutzerorganisation e.V., Karlsruhe (2014)
24. Roesch, M., et al.: Snort: lightweight intrusion detection for networks. In: LISA, vol. 99, pp. 229–238 (1999)